TA10: Spatio-Temporal Models in Stan

13/11/2026

Spatio-Temporal Models is Stan

Modelling spatio-temporal data in Bayesian statistics can be done by assuming the following model for a observations $Y(\mathbf{s}, t)$ observed at location $\mathbf{s}$ and at time t:

$$Y(\mathbf{s}, t) = \mathbf{X}(\mathbf{s}, t)^T \boldsymbol{\beta}(\mathbf{s}, t) + \mathbf{w}(\mathbf{s}, t) + \epsilon(\mathbf{s}, t),$$

where:

- $\mathbf{X}(\mathbf{s}, t)^T \boldsymbol{\beta}(\mathbf{s}, t)$ is the component that takes into account possible spatio-temporal covariates and regressors. According to the given covariates, simplified version of this regressive part are often considered;
- $\mathbf{w}(\mathbf{s}, t)$ is a spatio-temporal process. it can take several forms, according to the modelling assumptions;
- $\epsilon(\mathbf{s}, t)$ is a generic spatio-temporal white noise.

Monthly temperatures in Colorado

We consider a dataset taken from Banerjee, Carlin & Gelfand (2003) (https://www.taylorfrancis.com/books/mono/10.1201/9780203487808/hierarchical-modeling-analysis-spatial-data-sudipto-banerjee-sudipto-banerjee-bradley-carlin-alan-gelfand) about montly temperatures in Colorado.

In today's class, we will consider a way simpler example:

```
# Import modules
library(posterior)
library(bayesplot)
library(ggplot2)

# Import functions
library(cmdstanr)

# Create ./stan folder if does not exists
if (!dir.exists("./stan")) {
    dir.create("./stan")
}

df <- read.csv("colorado_spatio-temporal.csv", sep = " ")
```

```
head(df)
```

```
##   X.1      Lon    Lat     X   Jan  Feb  Mar  Apr  May  Jun  Jul  Aug  Sep  Oct
## 1   1 -109.330 37.870 2.156  0.07 0.22 1.08 1.14 2.08 2.53 2.83 2.61 2.27 1.53
## 2   2 -107.650 37.380 2.464  0.25 0.51 1.15 1.00 1.75 2.24 2.55 2.39 2.18 1.54
## 3   3 -106.657 36.956 2.561  0.02 0.36 1.20 1.01 1.71 2.27 2.56 2.37 2.15 1.37
## 4   4 -108.080 40.520 1.806 -0.02 0.25 1.10 1.18 2.14 2.71 2.96 2.84 2.34 1.64
## 5   5 -106.969 40.848 2.652 -0.22 0.07 0.68 0.67 1.43 2.01 2.29 2.13 1.73 1.03
## 6   6 -104.020 37.630 1.552  0.79 0.80 1.85 1.63 2.47 3.00 3.36 3.13 2.89 2.05
##    Nov   Dec
## 1 0.78  0.14
## 2 0.82  0.38
## 3 0.66  0.18
## 4 0.68  0.04
## 5 0.19 -0.04
## 6 1.03  0.51
```

We consider a separable spatio-temporal process, so that the spatial and temporal part can be modeled independently. We assume a 2D Gaussian Process in space and a AR(1) process in time. Hence, let $\mathbf{Y}_t = (Y(\mathbf{s}_1, t), \ldots, Y(\mathbf{s}_1, t)) \in \mathbb{R}^N$, and define $\mathbf{Y} = (\mathbf{Y}_1, \ldots, \mathbf{Y}_T) \in \mathbb{R}^{N \times T}$.

We specify the Bayesian model as:

$$
\begin{aligned}
\mathbf{Y} \mid \boldsymbol{\beta}, \boldsymbol{\alpha}, \mathbf{w}, \tau^2 &\sim \mathcal{N}(\mathbf{1}_{T \times 1} \otimes (\mathbf{X}\boldsymbol{\beta} + \mathbf{w}) + \boldsymbol{\alpha} \otimes \mathbf{1}_{N \times 1}, \tau^2 \mathbf{I}_{TN \times TN}) \\
\beta_p &\overset{\text{iid}}{\sim} \mathcal{N}(0, 5) \quad p = 1, \ldots, P \\
\boldsymbol{\alpha} \mid \sigma_\alpha^2 &\sim \mathcal{N}(\mathbf{0}, \sigma_\alpha^2 \mathbf{A}(\rho)) \\
\mathbf{w} \mid \sigma_w^2 &\sim \mathcal{N}(\mathbf{0}, \sigma_w^2 \mathbf{H}(\phi)) \\
\sigma_\alpha, \sigma_w, \tau &\overset{\text{iid}}{\sim} \mathcal{HC}(0, 1)
\end{aligned}
$$

With matrices defined as follows:

$$
\begin{aligned}
\mathbf{A}(\rho)_{ij} &= \frac{1}{(1 - \rho^2)} \rho^{|i-j|} \\
\mathbf{H}(\phi)_{ij} &= \exp\{-\phi |\mathbf{s}_i - \mathbf{s}_j|\}
\end{aligned}
$$

```
# Set problem dimensions
NSITES <- 50
NTIME <- 12
P <- 1

N <- NSITES * NTIME
```

```
# Useful functions for the spatio-temporal model
spt_tools <- "
// Computes Spatial covariance kernel
matrix cov_exp(matrix coords, real phi) {
    int N = dims(coords)[1];
    matrix[N,N] dist;
    for (i in 1:N) {
        for(j in i:N) {
            dist[i,j] = distance(coords[i], coords[j]);
            dist[j,i] = dist[i,j];
        }
    }
    return exp(-phi * dist);
}

// Computes Temporal covariance kernel
matrix cov_time(int T, real rho_t) {
    matrix[T,T] cov;
    for (i in 1:T) {
        for(j in i:T) {
            cov[i,j] = rho_t^abs(i-j);
            cov[j,i] = cov[i,j];
        }
    }
    return (1/(1-rho_t^2)) * cov;
}

// Computes kronecker product between vectors
vector kron_prod(vector A, vector B){
    int N = size(A);
    int P = size(B);
    vector[N*P] kron;
    for (i in 1:N) {
        kron[((i-1)*P+1):i*P] = A[i] * B;
    }
    return kron;
}
"

# Ensure the stan directory exists
if (!dir.exists("stan")) {
  dir.create("stan", recursive = TRUE)
}

# Write functions to file
stan_file <- "stan/spt-tools.stan"
writeLines(spt_tools, stan_file)
```

```
# GP-AR Spatio-Temporal Model with Stan
spt_lm <- "
functions {
    #include \"spt-tools.stan\"
}

data {
    int<lower=1> N;     // number of data points
    int<lower=1> T;     // number of time intervals
    int<lower=0> P;     // number of predictors
    matrix[N,T] Y;      // data
    matrix[N,P] X;      // matrix of predictors
    matrix[N,2] coords; // coordinates
    real<lower=0> phi;  // spatial range
    real<lower=0> rho;  // time innovation coefficient
}

transformed data {
    // Spatial covariance matrix
    matrix[N,N] H = cov_exp(coords, phi);

    // Temporal covariance matrix
    matrix[T,T] Ht = cov_time(T, rho);

    // Data vectorized
    vector[N*T] Y_vect = to_vector(Y);
}

parameters {
    vector[N] w;                // spatial process
    vector[T] alpha;            // temporal process
    vector[P] beta;             // predictor vectors
    real<lower=0> sigmas;       // spatial std. dev.
    real<lower=0> sigmat;       // temporal std. dev.
    real<lower=0> tau;          // linear std. dev.
}

transformed parameters {
    // Mean vector
    vector[N*T] mu = kron_prod(rep_vector(1,T), (X * beta + w)) + kron_prod(alpha, rep_vector(1,N));

    // Spatial Covariance Matrix
    matrix[N,N] Sigma_s = sigmas * sigmas * H;
    matrix[N,N] Lw = cholesky_decompose(Sigma_s);

    // Temporal Covariance Matrix
    matrix[T,T] Sigma_t = sigmat * sigmat * Ht;
    matrix[T,T] Lalpha = cholesky_decompose(Sigma_t);
}

model {
    // Likelihood
    Y_vect[1:N*T] ~ normal(mu[1:N*T], tau);

    // Prior
    w ~ multi_normal_cholesky(rep_vector(0,N), Lw);
    alpha ~ multi_normal_cholesky(rep_vector(0,T), Lalpha);
    beta ~ normal(0,5);
    sigmas ~ cauchy(0,1);
    sigmat ~ cauchy(0,1);
    tau ~ cauchy(0,1);
}
"

stan_file <- "stan/spt_lm.stan"
```

```
writeLines(spt_lm, stan_file)

spt_lm_model <- cmdstan_model(stan_file)
```

```
Y <- as.matrix(df[, 4:15])
dim(Y)
```

```
## [1] 50 12
```

```
X <- as.matrix(df$X - mean(df$X))
dim(X)
```

```
## [1] 50  1
```

```
coords <- as.matrix(df[, 2:3])
dim(coords)
```

```
## [1] 50  2
```

```
spt_lm_data <- list(
    N = NSITES,
    T = NTIME,
    P = P,
    Y = Y,
    X = X,
    coords = coords,
    phi = 1.09,
    rho = 0.9
)


spt_lm_fit <- spt_lm_model$sample(
    data = spt_lm_data,
    chains = 4,
    parallel_chains = 4,
    iter_sampling = 2000,
    refresh = 500,
    seed = 123,
    show_exceptions = F
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 3 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 1 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 4 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 2 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 2 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 3 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 3 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 1 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 1 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 4 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 4 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 2 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 1 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 2 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 3 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 1 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 4 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 2 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 2 finished in 38.8 seconds.
## Chain 3 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 3 finished in 39.5 seconds.
## Chain 4 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 4 finished in 39.8 seconds.
## Chain 1 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 1 finished in 40.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 39.6 seconds.
## Total execution time: 40.5 seconds.
```

```
posterior_spt_lm_fit <- posterior::as_draws_df(spt_lm_fit)
```

```
spt_lm_fit
```

```
##  variable   mean median   sd  mad     q5    q95 rhat ess_bulk ess_tail
##      lp__ 492.55 492.81 7.44 7.43 480.09 504.26 1.00     1823     2917
##      w[1]   0.07   0.08 0.07 0.07  -0.04   0.19 1.00     1584     3316
##      w[2]   0.11   0.11 0.07 0.07  -0.01   0.22 1.00     1176     2889
##      w[3]   0.15   0.14 0.08 0.08   0.02   0.27 1.00     1456     3361
##      w[4]  -0.12  -0.11 0.07 0.07  -0.23   0.00 1.00     1568     3549
##      w[5]  -0.11  -0.10 0.08 0.08  -0.23   0.02 1.00     1200     2870
##      w[6]   0.14   0.14 0.08 0.08   0.02   0.27 1.00     1831     3639
##      w[7]   0.10   0.09 0.07 0.07  -0.02   0.22 1.00     1201     2871
##      w[8]  -0.20  -0.20 0.08 0.08  -0.34  -0.07 1.00     2817     4757
##      w[9]  -0.04  -0.04 0.08 0.08  -0.18   0.09 1.00     1220     2671
##
##  # showing 10 of 5955 rows (change via 'max_rows' argument or 'cmdstanr_max_rows' option)
```

Homework

Let's learn the spatio-temporal process! Try to set proper priors to the parameters that defines the spatial and temporal covariance function, i.e., ϕ and ρ.

Remark: While dealing with spatio-temporal data, remember that parameters involved in the covariance function (ϕ for the spatial process and ρ for the temporal process) are non-identifiable. This non-identifiability is general to this type of models and may worse the quality of the sampling. The simplest solution is to fix those parameters or to set a more informative prior.

Kriging in spatio-temporal models

In spatial statistics, *kriging* is one of several methods that use a limited set of sampled data points to estimate the value of a variable over a continuous spatial field. In the Bayesian approach, this is nothing more than computing a posterior predictive distribution! In fact, if we want to estimate the value $y^* = y(\mathbf{s^*}, t)$ in a new location $\mathbf{s}^*$ for a given time t. Then, we just need to compute

$$\pi(y^* \mid \mathbf{Y}) = \int \mathcal{L}\left(y^* \mid w^*, \boldsymbol{\beta}, \boldsymbol{\alpha}, \tau^2\right) \cdot \pi\left(\mathrm{d}w^* \mid \mathbf{w}, \sigma_w^2\right) \cdot \pi\left(\mathrm{d}\boldsymbol{\beta}, \mathrm{d}\boldsymbol{\alpha}, \mathrm{d}\mathbf{w}, \mathrm{d}\sigma_w^2, \mathrm{d}\tau^2 \mid \mathbf{Y}\right), \tag{1}$$

where w^* is the spatial process in the new location $\mathbf{s}^*$. Now, call $\Sigma_{\mathbf{w}}(\phi) = \sigma_w^2 \mathbf{H}(\phi)$. Then:

$$\mathcal{L}\left(y^* \mid w^*, \boldsymbol{\beta}, \boldsymbol{\alpha}, \tau^2\right) = \mathcal{N}\left(y^* \mid \mathbf{x}^{*T}\boldsymbol{\beta} + \alpha_t + w^*, \tau^2\right), \tag{2}$$

$$\pi\left(w^* \mid \mathbf{w}, \sigma_w^2\right) = \mathcal{N}\left(w^* \mid \Sigma_{\mathbf{w},w^*}^T \Sigma_{\mathbf{w}}^{-1}\mathbf{w}, \Sigma_{w*} - \Sigma_{\mathbf{w},w^*}^T \Sigma_{\mathbf{w}}^{-1}\Sigma_{\mathbf{w},w^*}\right), \tag{3}$$

where $\Sigma_{\mathbf{w},w^*}$ is an $N \times 1$ matrix with elements $[\Sigma_{\mathbf{w},w^*}]_{i1} = \sigma_w^2 \exp\{-\phi|\mathbf{s}_i - \mathbf{s}^*|\}$. A similar computation can be done also in case of multiple prediction locations. Finally, we collect posterior draws for $\left(\boldsymbol{\beta}, \boldsymbol{\alpha}, \mathbf{w}, \sigma_w^2, \tau^2\right)$ from the MCMC. Hence, Kriging can be performed as follows: 1. Sample w^* from (3) after each posterior draws of $\left(\boldsymbol{\beta}, \boldsymbol{\alpha}, \mathbf{w}, \sigma_w^2, \tau^2\right)$; 2. Sample y^* given w^* and the posterior draws from (2); 3. Approximate (1) via Monte Carlo integration

```
spt_pred_tools <- "
// Computes spatial covariance kernel between observed and predicted coordinates
matrix cov_exp2(matrix coords1, matrix coords2, real phi) {
    int N1 = dims(coords1)[1];
    int N2 = dims(coords2)[1];
    matrix[N1,N2] dist;
    for (i in 1:N1) {
        for(j in 1:N2) {
            dist[i,j] = distance(coords1[i], coords2[j]);
        }
    }
    return exp(-phi * dist);
}

// Generates random sample from the posterior predictive distribution of a Gaussian Process
vector gp_pred_rng(vector W, matrix coords, matrix coords_pred, real sigma_sq, real phi) {

    // Setting dimensions
    int N1 = dims(coords)[1];
    int N2 = dims(coords_pred)[1];

    // Compute Covariance matrices
    matrix[N1,N1] Sigma11 = sigma_sq * cov_exp(coords, phi);
    matrix[N2,N2] Sigma22 = sigma_sq * cov_exp(coords_pred, phi);
    matrix[N1,N2] Sigma12 = sigma_sq * cov_exp2(coords, coords_pred, phi);

    // Buffers for output
    vector[N2] mu_pred;
    matrix[N2,N2] cov_pred;
    vector[N2] W_pred;

    // Helpers
    matrix[N1,N1] Sigma11_L;
    vector[N1] Sigma11_inv_W;
    matrix[N1,N2] L_inv_Sigma12;

    // Conditional mean
    Sigma11_L = cholesky_decompose(Sigma11);
    Sigma11_inv_W = mdivide_left_tri_low(Sigma11_L, W);
    Sigma11_inv_W = mdivide_right_tri_low(Sigma11_inv_W', Sigma11_L)';
    mu_pred = Sigma12' * Sigma11_inv_W;

    // Conditional covariance
    L_inv_Sigma12 = mdivide_left_tri_low(Sigma11_L, Sigma12);
    cov_pred = Sigma22 - L_inv_Sigma12' * L_inv_Sigma12;

    // Sample from posterior predictive distribution
    W_pred = multi_normal_rng(mu_pred, cov_pred);
    return W_pred;
}
"

stan_file <- "stan/spt_pred-tools.stan"
writeLines(spt_pred_tools, stan_file)
```

Notice the algebra behind the computation of conditional mean and variance of the Gaussian Process. The main idea is: *it is more efficient to invert a lower-triangular matrix w.r.t. a full matrix*. Hence, we rely on some algebra involving matrix operation and Cholesky factorization that relies only on inversion of lower triangular matrices.

Let $\mathbf{L}$ be the Cholesky factorization of $\Sigma_{\mathbf{w}}$. Then:

$$\Sigma_{\mathbf{w},w^*}^T \Sigma_{\mathbf{w}}^{-1} \mathbf{w} = \Sigma_{\mathbf{w},w^*}^T \left(\mathbf{L}\mathbf{L}^T\right)^{-1} \mathbf{w} = \Sigma_{\mathbf{w},w^*}^T \mathbf{L}^{-T} \mathbf{L}^{-1} \mathbf{w} = \Sigma_{\mathbf{w},w^*}^T \left[\left(\mathbf{L}^{-1}\mathbf{w}\right)^T \mathbf{L}^{-1}\right]^T;$$

$$\Sigma_{w*} - \Sigma_{\mathbf{w},w^*}^T \Sigma_{\mathbf{w}}^{-1} \Sigma_{\mathbf{w},w^*} = \Sigma_{w*} - \Sigma_{\mathbf{w},w^*}^T \left(\mathbf{L}\mathbf{L}^T\right)^{-1} \Sigma_{\mathbf{w},w^*} = \Sigma_{w*} - \Sigma_{\mathbf{w},w^*}^T \mathbf{L}^{-T} \mathbf{L}^{-1} \Sigma_{\mathbf{w},w^*} = \Sigma_{w*} - \left[\mathbf{L}^{-1}\Sigma_{\mathbf{w},w^*}\right]^T \left[\mathbf{L}^{-1}\Sigma_{\mathbf{w},w^*}\right].$$

```
spt_lm_pred <- "
functions {
    #include \"spt-tools.stan\"
    #include \"spt_pred-tools.stan\"
}

data {
    int<lower=1> N;               // number of data points
    int<lower=1> T;               // number of time intervals
    int<lower=0> P;               // number of predictors
    matrix[N,T] Y;                // data
    matrix[N,P] X;                // matrix of predictors
    matrix[N,2] coords;           // points coordinates
    real<lower=0> phi;            // spatial range
    real<lower=0> rho;            // time innovation coefficient
    int<lower=1> N_pred;          // number of data to predict
    matrix[N_pred,P] X_pred;      // matrix of predictors to pred
    matrix[N_pred,2] pred_coords; // coordinates of points to predict
    int<lower=1, upper=12> time;  // prediction time
}

transformed data {
    // Spatial covariance matrix
    matrix[N,N] H = cov_exp(coords, phi);

    // Temporal covariance matrix
    matrix[T,T] Ht = cov_time(T, rho);

    // Data vectorized
    vector[N*T] Y_vect = to_vector(Y);
}

parameters {
    vector[N] w;                // spatial process
    vector[T] alpha;            // temporal process
    vector[P] beta;             // predictor vectors
    real<lower=0> sigmas;       // spatial std. dev.
    real<lower=0> sigmat;       // temporal std. dev.
    real<lower=0> tau;          // linear std. dev.
}

transformed parameters {
    // Mean vector
    vector[N*T] mu = kron_prod(rep_vector(1,T), X * beta + w) + kron_prod(alpha, rep_vector(1,N));

    // Spatial Covariance Matrix
    matrix[N,N] Sigma_s = sigmas * sigmas * H;
    matrix[N,N] Lw = cholesky_decompose(Sigma_s);

    // Temporal Covariance Matrix
    matrix[T,T] Sigma_t = sigmat * sigmat * Ht;
    matrix[T,T] Lalpha = cholesky_decompose(Sigma_t);
}

model {
    // Likelihood
    Y_vect[1:N*T] ~ normal(mu[1:N*T], tau);

    // Prior
    w ~ multi_normal_cholesky(rep_vector(0,N), Lw);
    alpha ~ multi_normal_cholesky(rep_vector(0,T), Lalpha);
    beta ~ normal(0,5);
    sigmas ~ cauchy(0,5);
    sigmat ~ cauchy(0,5);
    tau ~ cauchy(0,5);
```

```
}

generated quantities {
    // Kriging
    array[N_pred] real Y_kriged;
    vector[N_pred] w_pred = gp_pred_rng(w, coords, pred_coords, (sigmas * sigmas), phi);
    vector[N_pred] mean_pred = X_pred * beta + w_pred + rep_vector(alpha[time], N_pred);
    Y_kriged[1:N_pred] = normal_rng(mean_pred[1:N_pred], tau);
}
"

stan_file <- "stan/spt_lm_pred.stan"
writeLines(spt_lm_pred, stan_file)

spt_lm_pred_model <- cmdstan_model(stan_file)
```

```
# Compute prediction box
x_min <- min(coords[, 1])
y_min <- min(coords[, 2])
x_max <- max(coords[, 1])
y_max <- max(coords[, 2])

# Compute prediction coordinates
x <- seq(x_min, x_max, length.out = 25)
y <- seq(y_min, y_max, length.out = 25)
grid <- expand.grid(x = x, y = y)
coords_pred <- as.matrix(grid)
dim(coords_pred)
```

```
## [1] 625   2
```

```
X_pred <- matrix(X[25, 1], nrow = nrow(coords_pred), ncol = 1)
dim(X_pred)
```

```
## [1] 625   1
```

```
# prediction month
month <- 1
```

```
spt_lm_pred_data <- list(
    N = NSITES,
    T = NTIME,
    P = P,
    Y = Y,
    X = X,
    coords = coords,
    phi = 1.09,
    rho = 0.9,
    N_pred = nrow(coords_pred),
    X_pred = X_pred,
    pred_coords = coords_pred,
    time = month
)

# Fit model via Stan
spt_lm_pred_fit <- spt_lm_pred_model$sample(
    data = spt_lm_pred_data,
    chains = 4,
    parallel_chains = 4,
    iter_sampling = 2000,
    refresh = 500,
    show_exceptions = F
)
```

```
## Running MCMC with 4 parallel chains...
##
## Chain 1 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 2 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 3 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 4 Iteration:    1 / 3000 [  0%]  (Warmup)
## Chain 1 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 4 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 3 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 2 Iteration:  500 / 3000 [ 16%]  (Warmup)
## Chain 1 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 4 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 1 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 4 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 3 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 2 Iteration: 1000 / 3000 [ 33%]  (Warmup)
## Chain 3 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 2 Iteration: 1001 / 3000 [ 33%]  (Sampling)
## Chain 1 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 4 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 3 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 2 Iteration: 1500 / 3000 [ 50%]  (Sampling)
## Chain 1 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 4 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 3 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 2 Iteration: 2000 / 3000 [ 66%]  (Sampling)
## Chain 1 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 2 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 3 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 4 Iteration: 2500 / 3000 [ 83%]  (Sampling)
## Chain 1 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 1 finished in 0.0 seconds.
## Chain 3 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 2 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 1 finished in 299.1 seconds.
## Chain 2 finished in 299.9 seconds.
## Chain 3 finished in 299.8 seconds.
## Chain 4 Iteration: 3000 / 3000 [100%]  (Sampling)
## Chain 4 finished in 300.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 299.7 seconds.
## Total execution time: 300.3 seconds.
```

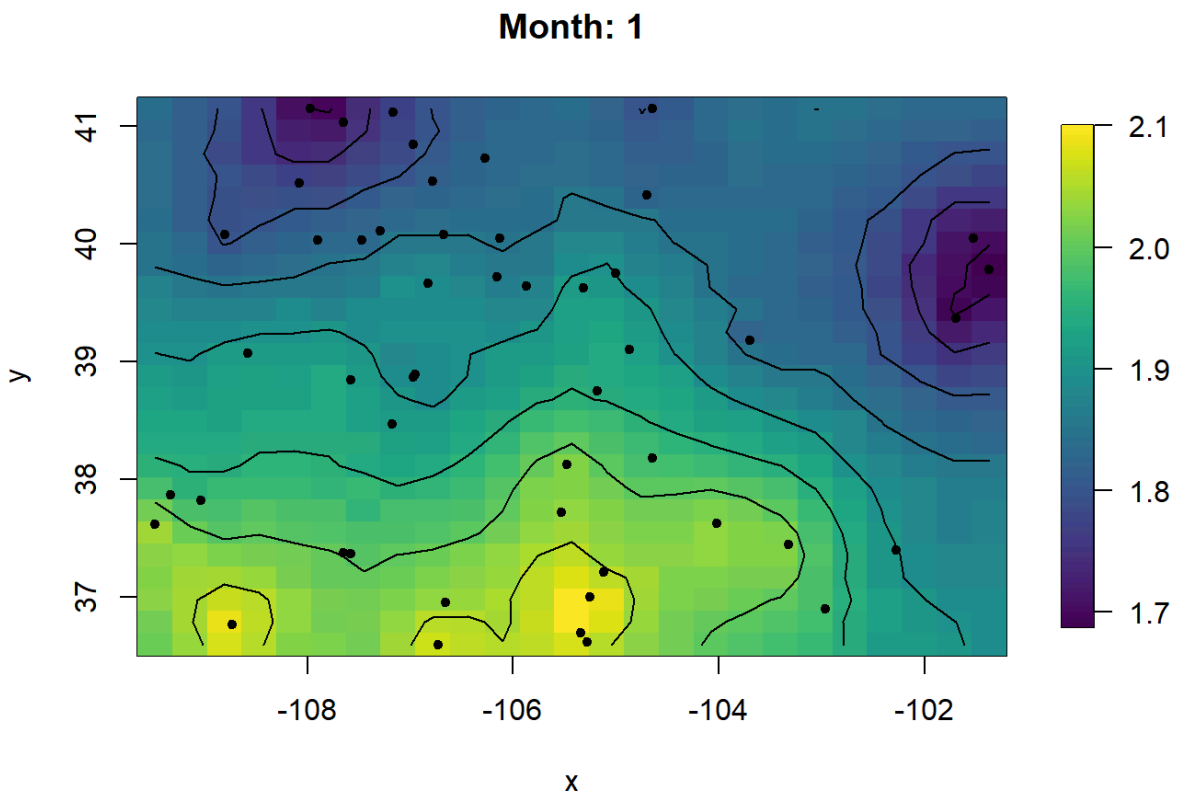
```
# Getting chain of kriged observations
Y_kriged_draws <- spt_lm_pred_fit$draws("Y_kriged", format = "matrix")

# Compute posterior mean and variance
Y_kriged_mean <- colMeans(Y_kriged_draws)
Y_kriged_var <- apply(Y_kriged_draws, 2, var)
```

```
library(viridis)
library(fields)


z_mean <- matrix(Y_kriged_mean, nrow = length(x), ncol = length(y))
z_var <- matrix(Y_kriged_var, nrow = length(x), ncol = length(y))

image.plot(x, y, z_mean, col = viridis(100), main = paste("Month:", month))
contour(x, y, z_mean, add = TRUE, col = "black", drawlabels = FALSE)
points(coords[, 1], coords[, 2], pch = 20, col = "black")
```




```
image.plot(x, y, z_var, col = viridis(100), main = paste("Month:", month))
contour(x, y, z_var, add = TRUE, col = "black", drawlabels = FALSE)
points(coords[, 1], coords[, 2], pch = 20, col = "black")
```

